Python Fundamentals for AI Engineering

A revision guide for Module 1. Plain explanations, one small example for each idea, and the traps worth remembering.

1	**Objects and data structures**	How Python really stores values
2	**Functions**	Arguments, scope, and the default-argument trap
3	**Iterators, generators, decorators**	Lazy data, closures, and wrapping functions
4	**Memory and concurrency**	Garbage collection, the GIL, threads and processes
5	**Asynchronous Python**	asyncio for many I/O calls at once
6	**Writing real code**	Modules, types, dataclasses, Pydantic, errors, JSON
7	**Testing and quality**	pytest, mocking, logging, Ruff
8	**Project setup**	Virtual environments, pyproject.toml, secrets

1
Objects and data structures

Everything in Python is an object, and variables are just labels pointing at objects.

Variables are labels, not boxes

When you write `b = a`, Python does not copy anything. It sticks a second label on the same object. So changing the object through one name shows up through the other.

```
a = [1, 2]
b = a            # same list, two names
b.append(3)
print(a)         # [1, 2, 3]
```

Watch out. Assignment never copies. Use `a.copy()` or `copy.deepcopy(a)` when you want a separate object.

Shallow copy vs deep copy

A shallow copy makes a new outer list but reuses the inner objects. A deep copy duplicates everything.

```
import copy
outer = [[1], [2]]
shallow = outer.copy()
deep = copy.deepcopy(outer)

outer[0].append(99)
print(shallow[0])   # [1, 99]  (shared inner list)
print(deep[0])      # [1]      (independent)
```

is vs ==

`==` asks "do these have the same value?". `is` asks "are these the exact same object?".

```
x = [1, 2]
y = [1, 2]
print(x == y)    # True
print(x is y)    # False
```

Watch out. Only use `is` for `None`: `if value is None`. Comparing numbers or strings with `is` can look like it works because of caching, then break.

Mutable vs immutable

Mutable objects can change in place: list, dict, set. Immutable objects cannot: int, float, str, tuple, frozenset. "Changing" an immutable value really creates a new object and moves the label.

> **Watch out.** A tuple is only shallowly immutable. `t = ([1], 2); t[0].append(9)` works, because the list inside is still mutable.

Hashing: why lists can't be dictionary keys

A dictionary finds a key by computing its hash, a number that tells it which bucket to look in. That only works if the hash never changes. A list can change, so Python refuses to hash it.

```
hash((13.08, 80.27))   # works: tuples are immutable
hash([1, 2])           # TypeError: unhashable type: 'list'
```

> **Watch out.** A tuple is hashable only if everything inside it is. Also, if you write `__eq__` on your own class without `__hash__`, the class becomes unhashable.

+= and + behave differently on lists

`+=` changes the list in place, so every name pointing at it sees the change. `+` builds a brand new list.

```
x = [1, 2, 3]
y = x
x += [4]       # in place: y is now [1, 2, 3, 4] too

x = [1, 2, 3]
y = x
x = x + [4]    # new list: y is still [1, 2, 3]
```

Choosing a data structure

Structure	Use it when	Speed to remember
list	You need an ordered collection you can change	Index and append are fast. Inserting or removing at the front is slow (O(n)).
tuple	A fixed record, like (13.08, 80.27)	Immutable. Can be a dictionary key.
dict	You look things up by key	Lookup is O(1) on average.
set	You need unique items or fast "is it in here?"	Membership is O(1) on average.
deque	A queue: add and remove at both ends	popleft is O(1). list.pop(0) is O(n) because every item shifts.

Average case vs worst case

The **worst case** is the most expensive input of a given size. The **average case** is the expected cost over a typical spread of inputs; it is not simply the mean of every possible case.

> **Watch out.** Dictionary lookup is O(1) on average but O(n) in the worst case, when many keys collide in the same bucket. Checking `x in some_list` inside a loop is slow; turn the list into a `set` first.

2
Functions

How arguments, scope and functions-as-objects work.

Positional, keyword and default arguments

```
def connect(host, port=5432):
    print(host, port)

connect("localhost")              # port uses its default
connect("localhost", 3306)        # positional
connect(port=3306, host="db")     # keyword, any order
```

> **Watch out.** Parameters with defaults must come after parameters without them. In a call, positional arguments come before keyword ones.

*args and **kwargs

`*args` gathers extra positional arguments into a **tuple**. `**kwargs` gathers extra keyword arguments into a **dict**. Only the stars matter; the names are a convention.

```
def run(model, *args, **config):
    print(model, args, config)

run("m1", 1, 2, temperature=0.7, top_p=0.9)
# m1 (1, 2) {'temperature': 0.7, 'top_p': 0.9}
```

Unpacking into a call

The stars also work the other way: `*` spreads a list or tuple into positional arguments, `**` spreads a dict into keyword arguments.

```
def box(a, b, c): return a * b * c

print(box(*[2, 3, 4]))          # 24

cfg = {"a": 2, "b": 3, "c": 4}
print(box(**cfg))               # 24
```

The mutable default argument trap

A default value is created **once**, when the function is defined, not on each call. If it is a list, every call shares it.

```
def add(item, items=[]):        # BUG
    items.append(item)
    return items

add("a")    # ['a']
add("b")    # ['a', 'b']   surprise

def add(item, items=None):      # FIX
    if items is None:
        items = []
    items.append(item)
    return items
```

> **Watch out.** This is the same reason a dataclass field cannot be `= []` and needs `field(default_factory=list)`.

Scope: the LEGB rule

Python looks up a name in this order: **L**ocal, **E**nclosing function, **G**lobal, **B**uilt-in. To change a variable from an outer function, declare it `nonlocal`.

```
def counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

next_id = counter()
next_id(); next_id()    # 1, then 2
```

Functions are objects

You can store a function in a variable, pass it to another function, or return it. That one idea powers decorators, callbacks and agent tools.

```
def square(x): return x * x

nums = [1, 2, 3, 4]
list(map(square, nums))                  # [1, 4, 9, 16]
list(filter(lambda x: x % 2 == 0, nums)) # [2, 4]
list(zip(["a", "b"], [90, 85]))          # [('a', 90), ('b', 85)]
```

> **Watch out.** map, filter and zip return lazy iterators in Python 3. Wrap them in `list()` to see the values.

3
Iterators, generators, decorators

Process data lazily, build functions that remember things, and wrap behaviour around functions.

Iterables and iterators

An **iterable** is anything you can loop over. An **iterator** hands out one value at a time and raises `StopIteration` when it runs out. A `for` loop is just `iter()` followed by repeated `next()`.

```
it = iter([10, 20, 30])
next(it)    # 10
next(it)    # 20
next(it)    # 30
next(it)    # StopIteration
```

Generators

A function that uses `yield` becomes a generator. It pauses at each `yield`, remembers where it was, and resumes on the next request. Nothing is built up front, so memory stays small even for huge data.

```
def read_batches(data, size):
    for i in range(0, len(data), size):
        yield data[i:i + size]

for batch in read_batches(list(range(10)), 4):
    print(batch)    # [0,1,2,3] [4,5,6,7] [8,9]
```

Watch out. A generator can be used only once. After it finishes, looping over it again gives nothing and raises no error.

Closures

A closure is an inner function that remembers variables from the function that created it, even after that outer function has returned.

```
def make_multiplier(n):
    def multiply(x):
        return x * n
    return multiply

double = make_multiplier(2)
double(5)    # 10
```

Watch out. Closures remember the **variable**, not its value at that moment. `[lambda: i for i in range(3)]` gives three functions that all return 2. Fix it with `lambda i=i: i`.

Decorators

A decorator takes a function and returns an upgraded version of it. Writing `@timer` above a function is exactly the same as `f = timer(f)`.

```
from functools import wraps
import time

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        print(func.__name__, time.perf_counter() - start)
        return result
    return wrapper

@timer
def add(a, b):
    return a + b
```

> **Watch out.** Always use `@wraps(func)` (it keeps the original name and docstring), accept `*args, **kwargs`, and return the result.

Decorators that take arguments

When the decorator itself needs settings, add one more layer: a function that returns the real decorator.

```
def retry(times=3):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except ConnectionError:
                    if attempt == times - 1:
                        raise
        return wrapper
    return decorator

@retry(times=3)
def call_api(): ...
```

> **Watch out.** FastAPI's `@app.get("/predict")` is a decorator too: it registers your function as a route.

Context managers

A `with` block sets something up and guarantees the cleanup, even if an error happens inside. Files, locks and database sessions all use it.

```
from contextlib import contextmanager

@contextmanager
def db_session():
    print("open")
    try:
        yield "connection"     # the with-body runs here
    finally:
        print("close")          # always runs

with db_session() as db:
    print("using", db)
```

Watch out. Without `try/finally` around the `yield`, the cleanup is skipped when the body raises. A class-based version uses `__enter__` and `__exit__`.

4

Memory and concurrency

How Python frees memory, and how to run things at the same time.

How memory is freed

CPython counts how many names point at each object. When the count reaches zero, the memory is freed immediately. A separate garbage collector handles objects that point at each other in a loop (a reference cycle).

```
a = [1, 2, 3]
b = a
del a          # removes the name 'a' only
print(b)        # still [1, 2, 3]
```

Watch out. Memory growth in a real app is usually something you are still holding on to: a global list, a cache with no limit, stored model outputs. The garbage collector cannot free anything that is still referenced.

The GIL

In standard CPython, the Global Interpreter Lock lets only one thread run Python bytecode at a time. So threads do not speed up CPU-heavy Python code, but they work well for programs that mostly wait on the network or disk. Libraries like NumPy and PyTorch release the lock during heavy native work.

> **Watch out.** The GIL does not make your code thread-safe. `count += 1` is really read, add, write, and two threads can interleave and lose an update.

Threads, processes or asyncio?

	Threads	Processes	asyncio
Best for	Waiting on I/O	Heavy CPU work in pure Python	Thousands of I/O waits
Memory	Shared	Separate	Shared, one thread
Cost	Low	Higher: startup, data copied between processes	Lowest
Tool	ThreadPoolExecutor	multiprocessing	asyncio

Rule of thumb: waiting on APIs, databases or files means threads or asyncio. Crunching numbers in pure Python means processes. Tensor math means a GPU framework.

Race conditions and locks

When two threads change the same data, results can be wrong. A lock lets only one thread through a section at a time.

```
import threading
count = 0
lock = threading.Lock()

def work():
    global count
    for _ in range(100_000):
        with lock:        # one thread at a time
            count += 1
```

Running I/O work in threads

ThreadPoolExecutor is the simplest way to make many I/O calls at once.

```
from concurrent.futures import ThreadPoolExecutor

def call_model(prompt):
    return "answer for " + prompt

with ThreadPoolExecutor(max_workers=5) as pool:
    results = list(pool.map(call_model, ["q1", "q2", "q3"]))
```

> **Watch out.** With multiprocessing, wrap your entry point in `if __name__ == "__main__":` and only pass objects that can be pickled.

5
Asynchronous Python

Wait for many things at once without blocking. The key skill for calling LLM APIs.

The core idea

While one request waits for a response, the program can start others. Three sequential 2-second calls take about 6 seconds. Run them concurrently and they take about 2.

```
import asyncio

async def fetch(name):
    await asyncio.sleep(2)      # stands in for an API call
    return name

async def main():
    results = await asyncio.gather(fetch("A"), fetch("B"), fetch("C"))
    print(results)              # ['A', 'B', 'C'] after ~2 seconds

asyncio.run(main())
```

> **Watch out.** Results from `gather` come back in the order you passed them, not the order they finished.

The pieces

async def creates a coroutine function. Calling it returns a coroutine object and runs nothing yet.
await pauses the current coroutine so the **event loop** can run others.
Task is a coroutine scheduled on the loop with `create_task`.
Future is a placeholder for a result that arrives later. A Task is a kind of Future.
asyncio.run(main()) starts the loop. Call it once, at the edge of your program.

Limit concurrency and add timeouts

Firing 1,000 requests at once hits rate limits. A semaphore caps how many run together, and a timeout stops a slow call from hanging.

```
sem = asyncio.Semaphore(5)              # at most 5 at a time

async def call_llm(prompt):
    async with sem:
        async with asyncio.timeout(3):  # TimeoutError after 3 s
            await asyncio.sleep(1)
            return "answer: " + prompt

results = await asyncio.gather(
    *(call_llm(p) for p in prompts),
    return_exceptions=True,             # one failure won't cancel the rest
)
```

Never block the event loop

A normal blocking call inside async code freezes every task until it returns.

```
async def bad():
    time.sleep(5)                # freezes EVERYTHING for 5 s

async def good():
    await asyncio.sleep(5)       # others keep running

async def blocking_library():
    # run a sync function in a worker thread instead
    return await asyncio.to_thread(read_big_file)
```

Four more traps

1. Forgetting await. You get a coroutine object and a warning, and the work never runs.
2. gather without return_exceptions. The first error is raised immediately and the other tasks keep running in the background.
3. Losing a task. Keep a reference to anything from `create_task`, or it may be garbage-collected mid-run.
4. Nesting asyncio.run(). Inside a running loop (Jupyter, FastAPI) just `await` the coroutine.

6
Writing real code

Structure, types, validation and error handling for maintainable programs.

Modules and packages

A **module** is a .py file. A **package** is a folder of modules, usually with an `__init__.py`. Python runs a module's top-level code once, the first time it is imported.

```
# main.py
from calculations import add

def main():
    print(add(10, 20))

if __name__ == "__main__":   # runs only when executed directly
    main()
```

Watch out. Don't open connections or load models at the top level of a module; they run at import time. If A imports B and B imports A you get a circular import error: move the shared code to a third module.

Type hints

Type hints document what a function expects and returns, and let editors and tools like mypy catch mistakes.

```
def average(a: float, b: float) -> float:
    return (a + b) / 2

def find_email(user_id: int) -> str | None:   # may return None
    ...

names: list[str] = ["a", "b"]
scores: dict[str, float] = {"math": 95.5}
```

Watch out. Type hints are **not enforced at runtime**. Passing a string to `average` still runs.

Generics with TypeVar

A TypeVar keeps the input and output types connected, so a checker knows that a list of ints gives back an int.

```
from typing import TypeVar
T = TypeVar("T")

def first(items: list[T]) -> T:
    return items[0]

first([1, 2, 3])        # int
first(["a", "b"])       # str
```

Dataclasses

A dataclass writes `__init__`, `__repr__` and `__eq__` for you. Use it for plain internal data.

```
from dataclasses import dataclass, field

@dataclass(frozen=True)          # frozen = fields can't be reassigned
class ModelConfig:
    name: str
    temperature: float = 0.7
    tags: list[str] = field(default_factory=list)   # never `= []`
```

Watch out. `frozen=True` blocks reassigning fields, but a list stored inside can still be changed.

Pydantic

Pydantic checks data when it enters your program: API requests, JSON files, LLM output. It converts what it safely can and raises an error for the rest.

```
from pydantic import BaseModel, Field

class ModelConfig(BaseModel):
    model_name: str
    temperature: float = Field(ge=0, le=2)
    max_tokens: int = Field(gt=0, le=10000)

ModelConfig(model_name="m", temperature="0.7", max_tokens=100)
# "0.7" is converted to 0.7.   "hot" would raise ValidationError.
```

Watch out. ge is "greater or equal", le is "less or equal", gt and lt are strict.

Dataclass or Pydantic?

	Dataclass	Pydantic
Use it for	Internal data you already trust	Data coming from outside
Validates?	No	Yes, with conversion and constraints
Typical use	Domain objects	API schemas, settings, structured LLM output

Handling errors

Catch the specific error you expect. Define your own exception classes for your app, and use `raise ... from e` to keep the original cause.

```
class ConfigError(Exception):
    pass

try:
    data = json.load(open(path, encoding="utf-8"))
except FileNotFoundError as e:
    raise ConfigError("config file missing") from e
except json.JSONDecodeError as e:
    raise ConfigError("config is not valid JSON") from e
else:
    print("loaded")        # only if nothing failed
finally:
    print("done")          # always runs
```

Watch out. A bare `except:` also swallows Ctrl+C. `except Exception: pass` hides real bugs. A `return` inside `finally` silently discards an in-flight error.

Files and JSON

Always read and write files inside a `with` block so they close properly.

Function	What it does
json.dumps(obj)	Python object to a JSON string
json.loads(text)	JSON string to a Python object
json.dump(obj, file)	Write JSON to a file
json.load(file)	Read JSON from a file

7
Testing and quality

Prove your code works, find out why it doesn't, and keep it tidy.

Your first pytest tests

pytest finds files named `test_*.py` and functions named `test_*`. A test passes when its `assert` is true.

```
def test_add():
    assert add(10, 20) == 30

def test_empty_list_rejected():
    with pytest.raises(ValueError):
        average([])
```

Watch out. Test the edges as well as the happy path: empty input, None, boundaries, wrong types, timeouts. Use `assert` in tests only; Python removes asserts when run with `-O`.

Many inputs, one test

parametrize runs the same test against a table of inputs.

```
@pytest.mark.parametrize("a, b, expected", [
    (2, 3, 5),
    (0, 0, 0),
    (-2, 5, 3),
])
def test_add(a, b, expected):
    assert add(a, b) == expected
```

Fixtures

A fixture is reusable setup. Name it as a test parameter and pytest injects it. Code after `yield` runs as cleanup.

```
@pytest.fixture
def documents():
    docs = [{"id": 1, "text": "Python"}]
    yield docs
    # cleanup would go here

def test_count(documents):
    assert len(documents) == 1
```

Watch out. Scopes: function (default), class, module, package, session.

Mocking external services

Real LLM calls are slow, cost money and give different answers each time. In unit tests, replace them with a fake.

```
from unittest.mock import patch

def test_generate():
    with patch("ai_service.call_llm_api", return_value="mock") as m:
        assert generate_response("hi") == "mock"
        m.assert_called_once_with("hi")
```

Watch out. Patch the name **where it is used**, not where it is defined. If ai_service.py does `from client import call_llm_api`, patch `ai_service.call_llm_api`. `side_effect=SomeError()` makes the mock raise.

Kinds of tests

Type	Checks	Example
Unit	One function	clean_query() strips spaces
Integration	Parts working together	Retrieval returns documents
End-to-end	The whole workflow	Question in, answer out
Regression	Old behaviour still works	A fixed bug stays fixed

TDD has three steps: write a failing test (red), write just enough code to pass (green), then clean up (refactor).

Debugging and logging

Use `breakpoint()` to pause and inspect: `n` next line, `s` step into, `c` continue, `p x` print x, `q` quit. For anything running on a server, use logging instead of print.

```
import logging
logger = logging.getLogger(__name__)

try:
    answer = generate(prompt)
    logger.info("generated answer for user %s", user_id)
except Exception:
    logger.exception("generation failed")   # includes the traceback
    raise
```

Watch out. Never log API keys, tokens or sensitive prompts.

Log levels

Level	Use for
DEBUG	Detail for diagnosing problems
INFO	Normal events
WARNING	Odd, but it carries on
ERROR	An operation failed
CRITICAL	The app can't continue

Keeping code tidy

Ruff is a fast linter and formatter. A good habit before every commit:

```
ruff check . --fix     # find and fix issues
ruff format .          # consistent formatting
mypy src               # type checking
pytest                 # run the tests
```

8

Project setup

Isolated environments, dependency files and safe handling of secrets.

Virtual environments

Each project gets its own isolated set of packages, so versions from different projects never clash.

```
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install fastapi pytest
deactivate
```

Watch out. If you forget to activate the environment, packages install globally. Add `venv/`, `__pycache__/`, `.pytest_cache/` and `.env` to `.gitignore`.

requirements.txt and pip

A requirements file records exact package versions so someone else can rebuild your environment.

```
pip freeze > requirements.txt     # save versions
pip install -r requirements.txt   # recreate elsewhere
```

Watch out. `pip freeze` also lists packages your packages depend on, so it is a snapshot, not a list of what you chose. Keep your direct dependencies in `pyproject.toml`.

pyproject.toml

The modern, single configuration file for a Python project: its name, Python version, dependencies, and tool settings.

```
[project]
name = "ai-utils"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = ["pydantic>=2.0"]

[project.optional-dependencies]
dev = ["pytest", "ruff", "mypy"]
```

Install it in editable mode with dev tools using `pip install -e ".[dev]"`. Editable means your source edits take effect immediately. `python -m build` creates a distributable package.

Secrets and configuration

Never put an API key in your code. Read it from the environment, keep local values in a `.env` file that git ignores, and load them with pydantic-settings so missing values fail at startup.

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    openai_api_key: str          # required, app won't start without it
    database_url: str
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

settings = Settings()
```

> **Watch out.** If a real key is ever committed, it stays in git history. Deleting the file is not enough; rotate the key.

A clean project layout

Separate the web layer, business logic, settings and data models so you can change one without touching the others.

```
project/
  src/
    my_app/
      __init__.py
      api/          HTTP routes
      services/     business logic (LLM, retrieval)
      core/         settings
      models/       schemas
      utils/        small helpers
  tests/
  pyproject.toml
  .gitignore
  README.md
```

> **Watch out.** Swapping an LLM or embedding provider should only touch one file in services/.

What comes next

HTTP and REST, **FastAPI** (routing, dependency injection, middleware), Pydantic API schemas, **PostgreSQL and SQLAlchemy**, JWT authentication, **Redis**, Docker and backend testing. First project: an AI Model Management API.